Fall 2026 Exploratory Data Analysis with R

Lecture 8: Advanced Plots and Regression I

Brian Robertson
Federal Reserve Bank of Atlanta
2026-10-23

01

Recap & Agenda

01 Recap & Agenda
02 The Grammar of Graphics
03 Beautifying Plots
04 Looking for Relationships: The Redfin Data
05 Regression I
06 Economic vs. Statistical Significance
07 Research Process & The Project
08 Recap

Quick Recap: Last Week

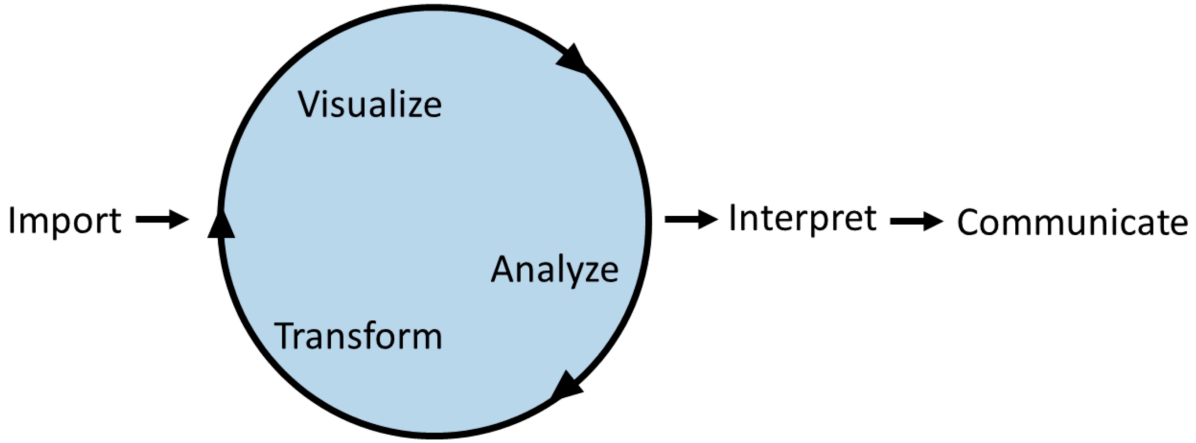
- **Loops** automate a repeated task over a set, using an index and a block of code run once per element
- **Debugging** is a skill — recognize the bug category (syntax, violated assumptions, logic, silly mistakes), then isolate it with `print()` or `browser()`

Today: back to statistics, with a visual and a modeling toolkit. We'll push our **plots** further than `aes()` and a single `geom_*()`, then take our first look at **regression** — a tool for summarizing a relationship with a line.

Goals for Today

- Review the **Grammar of Graphics** and use it to build up a plot step by step
- **Beautify** a plot with `labs()`, pre-made themes, and scale adjustments
- Explore a new dataset — Redfin home sale listings — using correlation and scatterplots
- Fit a simple linear **regression** with `lm()`, and pull its results the tidy way with `broom`
- Distinguish **statistical significance** from **economic significance**
- Connect today's tools to **Project Update #6**

The Big Picture



- Visualization and regression both sit inside the same loop — explore, transform, model, communicate, and back again
- Today adds two more tools to that loop: more expressive plots, and a first model

02

The Grammar of Graphics

01 Recap & Agenda
02 The Grammar of Graphics
03 Beautifying Plots
04 Looking for Relationships: The Redfin Data
05 Regression I
06 Economic vs. Statistical Significance
07 Research Process & The Project
08 Recap

Review: The Grammar of Graphics (Lecture 5)

Every `ggplot` breaks down into three elements:

- **Data** — the underlying data points
- **Mapping** — which **aesthetic** (x/y axis, color, size, …) a variable is assigned to, inside `aes()`
- **Geometry** — the visual form that represents the data (`geom_point()`, `geom_line()`, `geom_col()`, …)



Today we push past the basics: more aesthetics, more geometries, and a plot that's ready to show someone other than yourself.

New Data: Median Weekly Earnings

BLS data on median weekly earnings by age group, 2010–2020.

```
glimpse(median_earnings)
```

```
Rows: 352
Columns: 3
$ date               <date> 2010-01-01, 2010-04-01, 2010-07-01, 2010-10-01, 20…
$ age                <chr> "16 to 19 years", "16 to 19 years", "16 to 19 years…
$ median_weekly_earn <dbl> 356, 346, 335, 360, 360, 349, 356, 346, 362, 357, 3…
```

- How many age groups are there? What would happen if we tried to plot all of them in one `ggplot()` call without thinking about aesthetics first?

Building a Plot, One Step at a Time

1. Initialize a `ggplot` object with your **data**
2.
3.

```
median_earnings %>%
  filter(age == "25 to 34 years") %>%
  ggplot()
```

Building a Plot, One Step at a Time

1. Initialize a `ggplot` object with your data
2. Map a variable to an aesthetic
3.

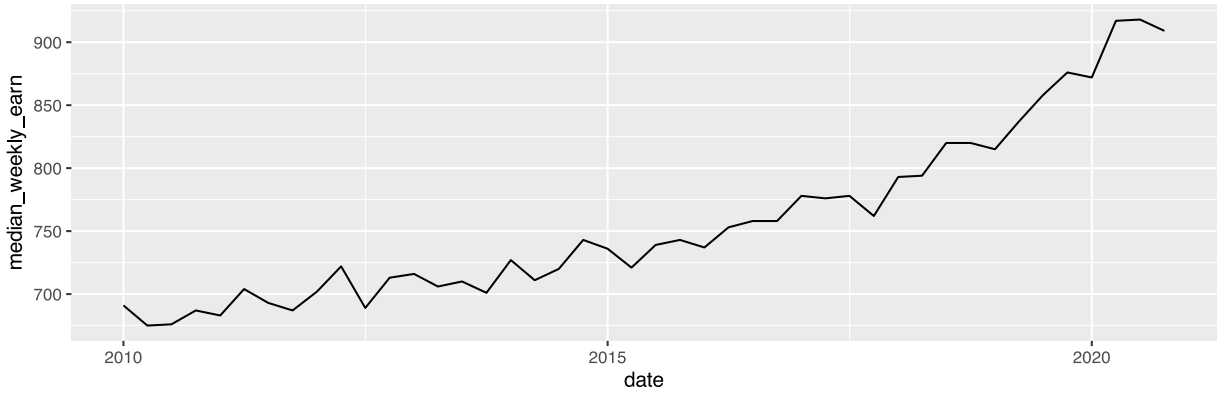
```
median_earnings %>%
  filter(age == "25 to 34 years") %>%
  ggplot(aes(x = date, y = median_weekly_earn))
```

- Nothing is drawn yet — `aes()` only sets up the mapping, it doesn't pick a visual form

Building a Plot, One Step at a Time

1. Initialize a `ggplot` object with your data
2. Map a variable to an aesthetic
3. Add a layer of geometry

```
median_earnings %>%
  filter(age == "25 to 34 years") %>%
  ggplot(aes(x = date, y = median_weekly_earn)) +
  geom_line()
```

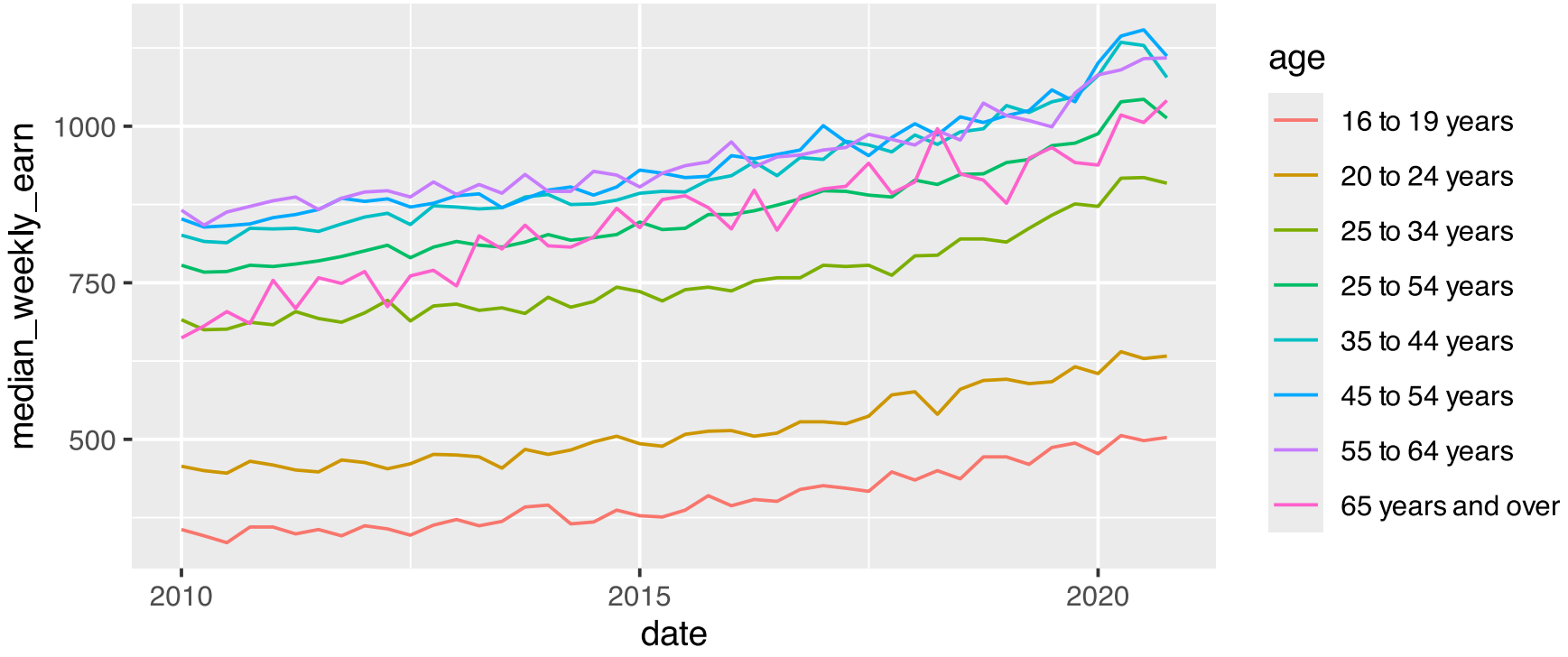


Comparing Across Groups

- We have 8 age groups. Filtering to one at a time, and copy-pasting the plot code eight times, is exactly the kind of repetition a loop (or a smarter aesthetic mapping) should replace
- How do we put every age group on one plot?

Map `age` to the **color** aesthetic — inside `aes()`, since it comes from the data:

```
median_earnings %>%
  ggplot(aes(x = date, y = median_weekly_earn, color = age)) +
  geom_line()
```

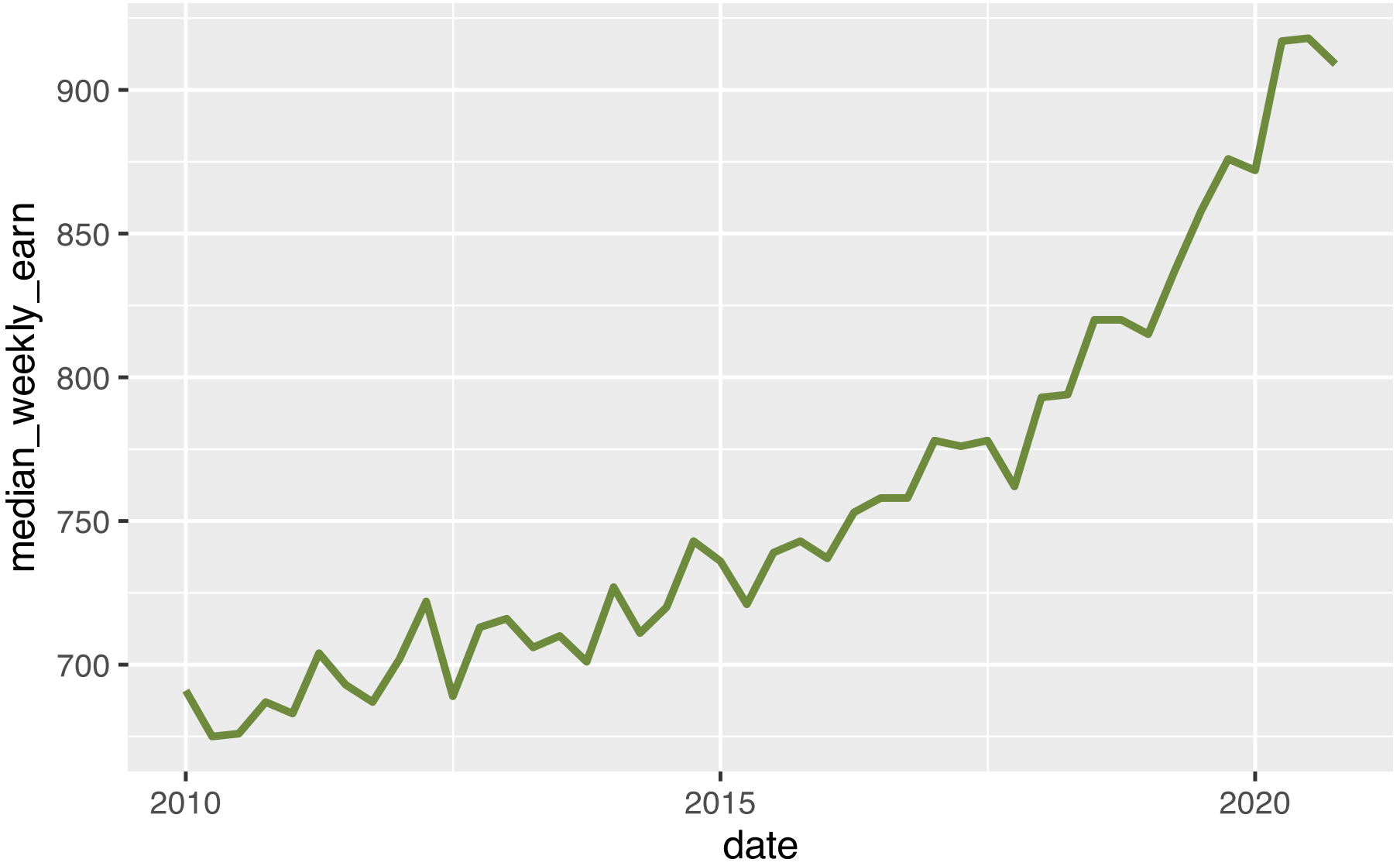


Why Does `aes()` Matter Here?

- `aes(color = age)` tells `ggplot` to look in the data for the `age` column and map it to color — one line per age group, one color per line
- Without it, `ggplot` has nowhere to look for `age` and does nothing useful with it

Compare: setting a visual property **manually**, outside `aes()`, sets it to one fixed value for everything — no mapping, no legend:

```
median_earnings %>%
  filter(age == "25 to 34 years") %>%
  ggplot(aes(x = date, y = median_weekly_earn)) +
  geom_line(color = "darkolivegreen4", linewidth = 1)
```


900
850
800
750
700
median_weekly_earn
2010
2015
2020
date

`aes()`: Inside vs. Outside

- **Inside `aes()`** — "map this *column* to this aesthetic." The values come from the data and can vary row to row (a line per age group, a color per category).
- **Outside `aes()`, as a plain argument** — "just set this property to one fixed value." No column lookup, no legend.

A common bug: putting a fixed value *inside* `aes()`, or a column name *outside* it. Both run without an error — but the plot is wrong, with no warning that anything's off.

Breakout: Gapminder (10 minutes)

The `gapminder` package ships a dataset with life expectancy and GDP per capita for every country, 1952–2007.

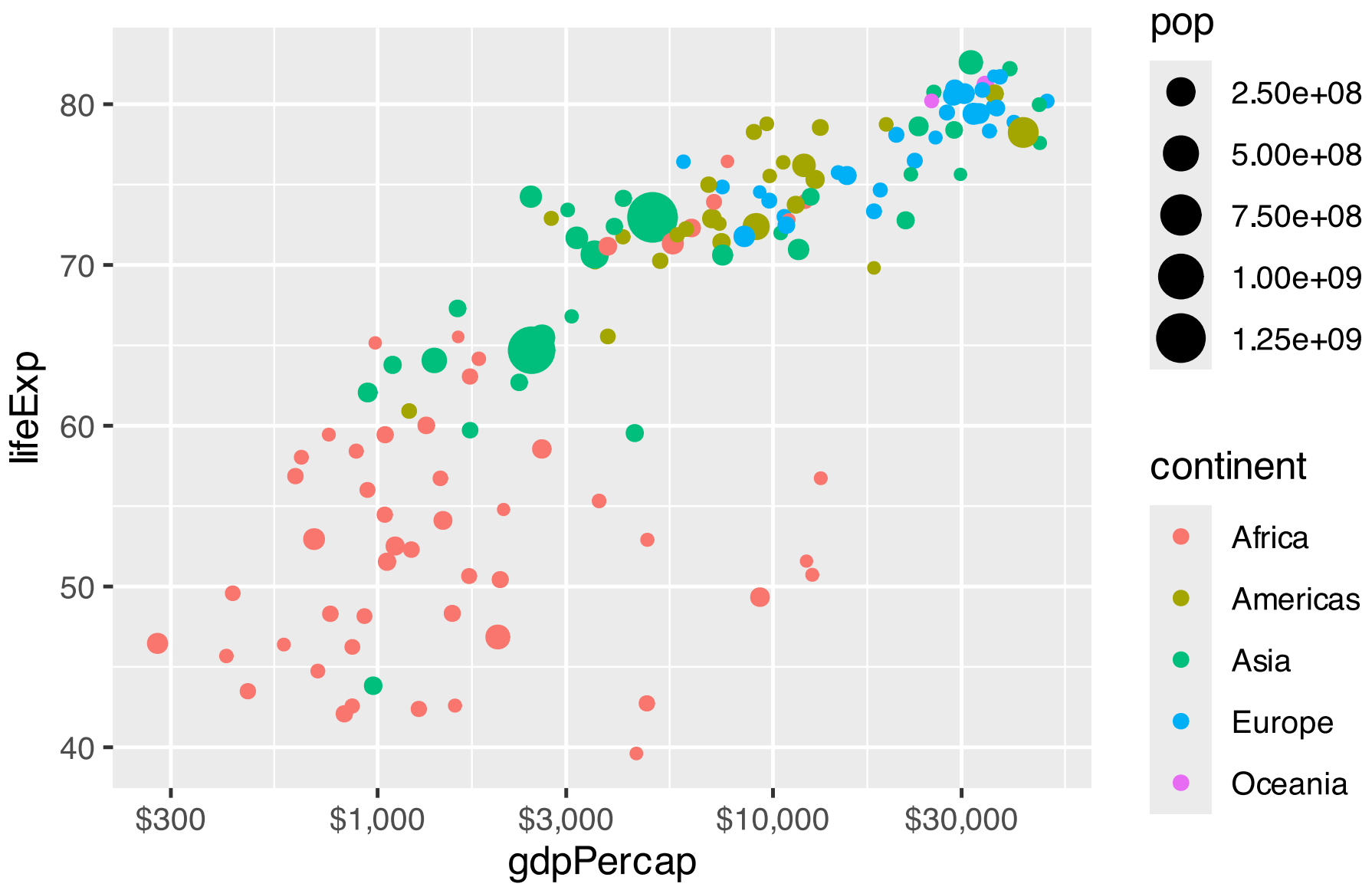
```
glimpse(gapminder)
```

```
Rows: 1,704
Columns: 6
$ country   <fct> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", …
$ continent <fct> Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, …
$ year      <int> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, …
$ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8…
$ pop       <int> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12…
$ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, …
```

1. Filter to `year == 2007`.
2. Map GDP per capita to the x-axis, life expectancy to the y-axis, continent to color, and population to size.
3. Represent the data with the point geometry.
4. **Challenge:** make the x-axis a log scale (hint: a `scale_x_*` function).

Solution

```
gapminder %>%
  filter(year == 2007) %>%
  ggplot(aes(x = gdpPercap, y = lifeExp, color = continent, size = pop)) +
  geom_point() +
  scale_x_log10(labels = scales::dollar_format(accuracy = 1))
```


lifeExp
80
70
60
50
40
$300
$1,000
$3,000
$10,000
$30,000
gdpPercap
pop
2.50e+08
5.00e+08
7.50e+08
1.00e+09
1.25e+09
continent
Africa
Americas
Asia
Europe
Oceania

Discussion

- We mapped four variables — GDP per capita, life expectancy, continent, and population — onto one plot with a single `geom_point()` call
- What patterns do you see? Is the relationship between GDP per capita and life expectancy a straight line?
- Where does the data not behave the way the overall pattern suggests (outliers, within-continent variation)?

03

Beautifying Plots

01 Recap & Agenda
02 The Grammar of Graphics
03 Beautifying Plots
04 Looking for Relationships: The Redfin Data
05 Regression I
06 Economic vs. Statistical Significance
07 Research Process & The Project
08 Recap

Why Beautify a Plot?

A clear, well-labeled plot:

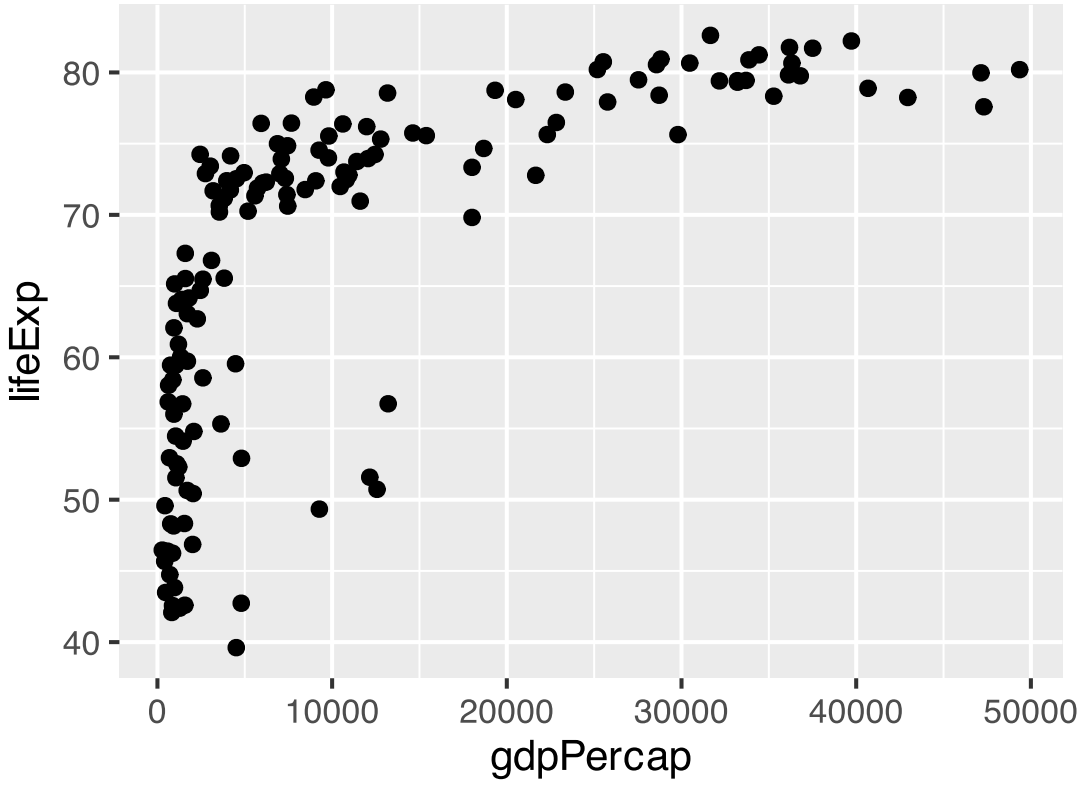
- Communicates its main message quickly, without the reader doing extra work
- Lets a careful reader dig into the detail
- Looks professional enough that someone will trust it

A chart only you will ever see can stay rough. A chart you hand to your adviser, your classmates, or a reader of your final paper should not.

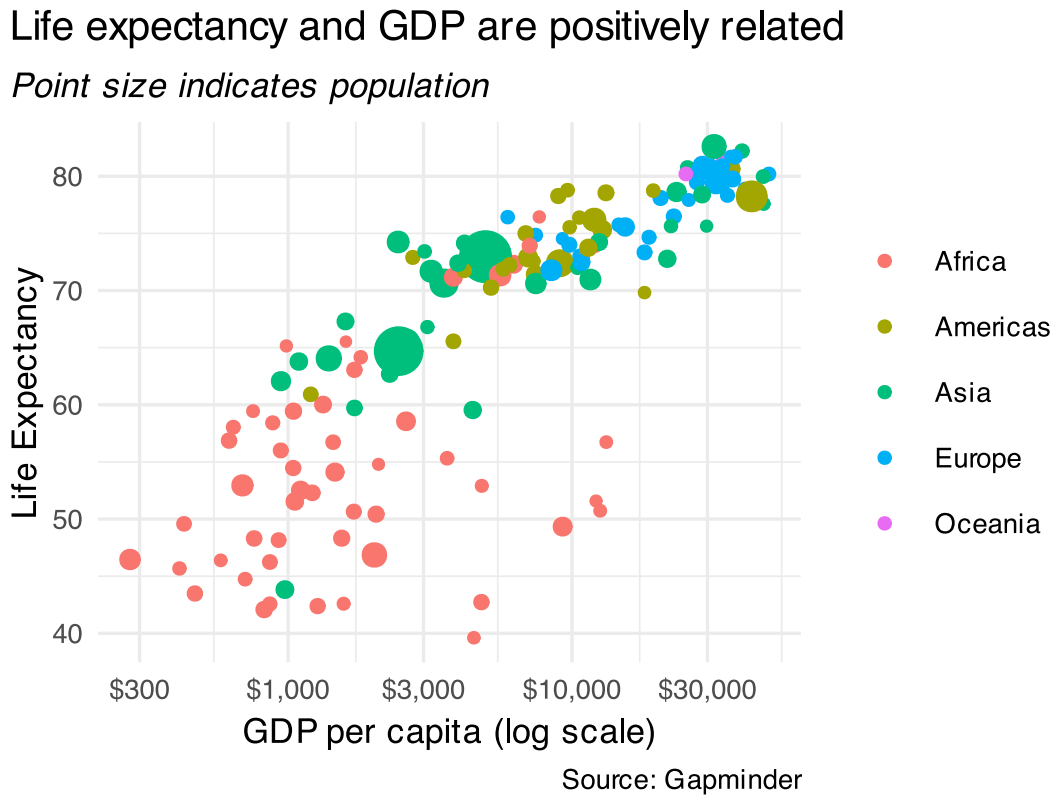
The Same Data, Two Ways

```
gapminder %>%
  filter(year == 2007) %>%
  ggplot(aes(gdpPercap, lifeExp)) +
  geom_point()
```

Default — no labels, no color, default theme:



Beautified — labeled, mapped, themed:



Same data, same geometry — the right side just mapped two more variables and spent a few extra lines on labels and a theme.

Adding Titles, Subtitles, and Captions: `labs()`

`labs()` sets a title, subtitle, caption, axis titles, and legend titles — all in one call:

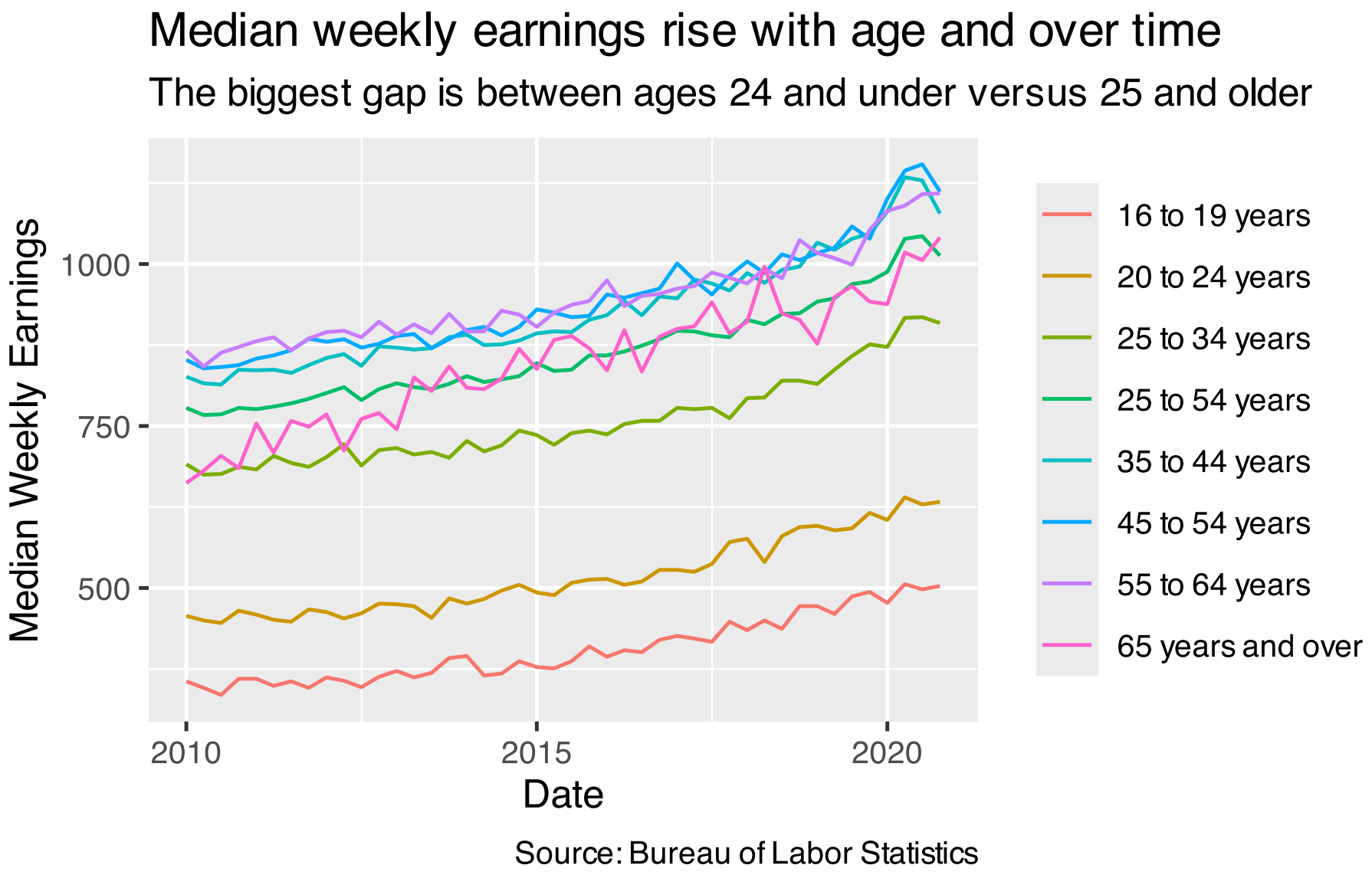
```
labs(title = "Median weekly earnings rise with age and over time",
     subtitle = "The biggest gap is between ages 24 and under versus 25 and older",
     caption = "Source: Bureau of Labor Statistics",
     x = "Date", y = "Median Weekly Earnings",
     color = NULL)
```

- `color = NULL` removes the legend title — useful when the plot title already makes clear what the colors mean

Our Plot With Labels

```
age_plot <- median_earnings %>%
  ggplot(aes(date, median_weekly_earn, color = age)) +
  geom_line() +
  labs(title = "Median weekly earnings rise with age and over time",
       subtitle = "The biggest gap is between ages 24 and under versus 25 and older",
       caption = "Source: Bureau of Labor Statistics",
       x = "Date", y = "Median Weekly Earnings",
       color = NULL)

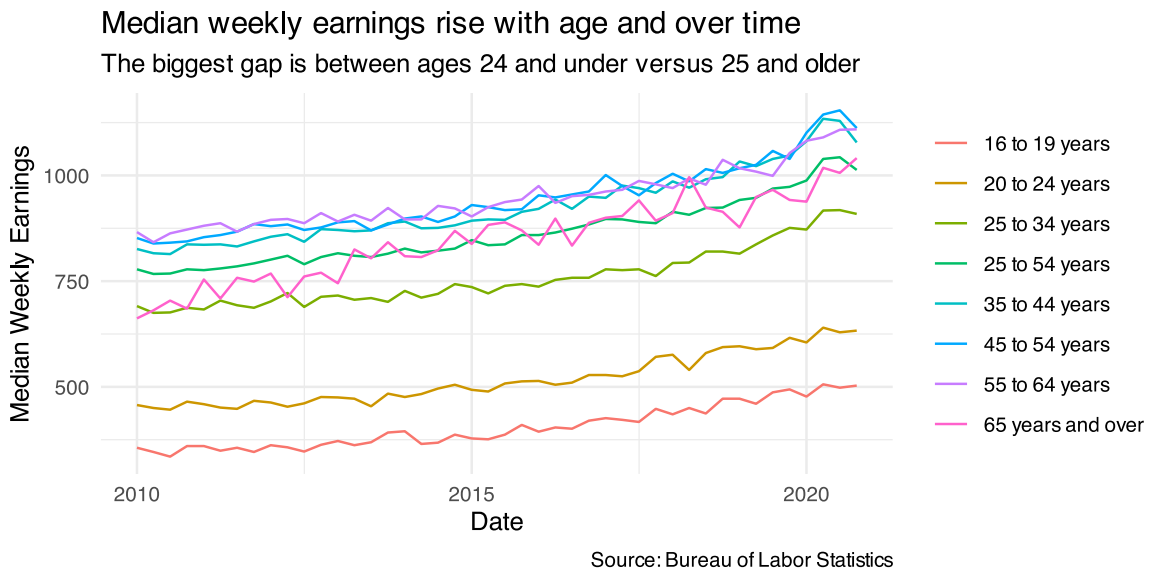
age_plot
```


Median weekly earnings rise with age and over time
The biggest gap is between ages 24 and under versus 25 and older
Median Weekly Earnings
1000
750
500
2010
2015
2020
Date
16 to 19 years
20 to 24 years
25 to 34 years
25 to 54 years
35 to 44 years
45 to 54 years
55 to 64 years
65 years and over
Source: Bureau of Labor Statistics

Using Pre-Made Themes

A single `theme_*()` call changes many things at once — background, grid lines, legend position:

```
age_plot <- age_plot + theme_minimal()
age_plot
```



- The `ggthemes` package has more, including `theme_economist()`

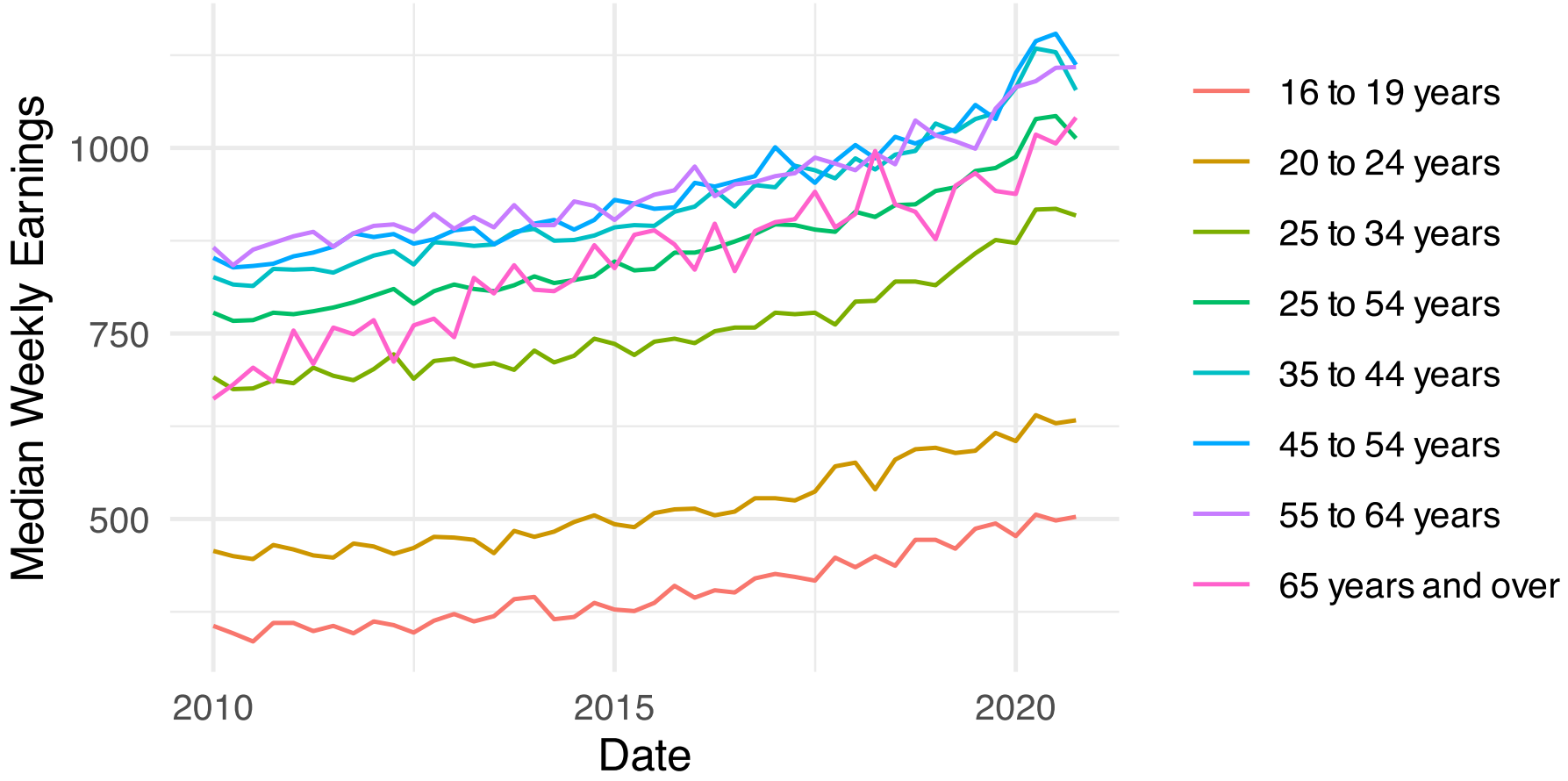
Making Your Own Theme Adjustments

Layer `theme()` on top of a pre-made theme to tweak individual elements — here, left-aligning the title and italicizing the subtitle:

```
age_plot <- age_plot +
  theme(plot.title.position = "plot",
        plot.subtitle = element_text(face = "italic", hjust = 0))
age_plot
```

Median weekly earnings rise with age and over time

The biggest gap is between ages 24 and under versus 25 and older

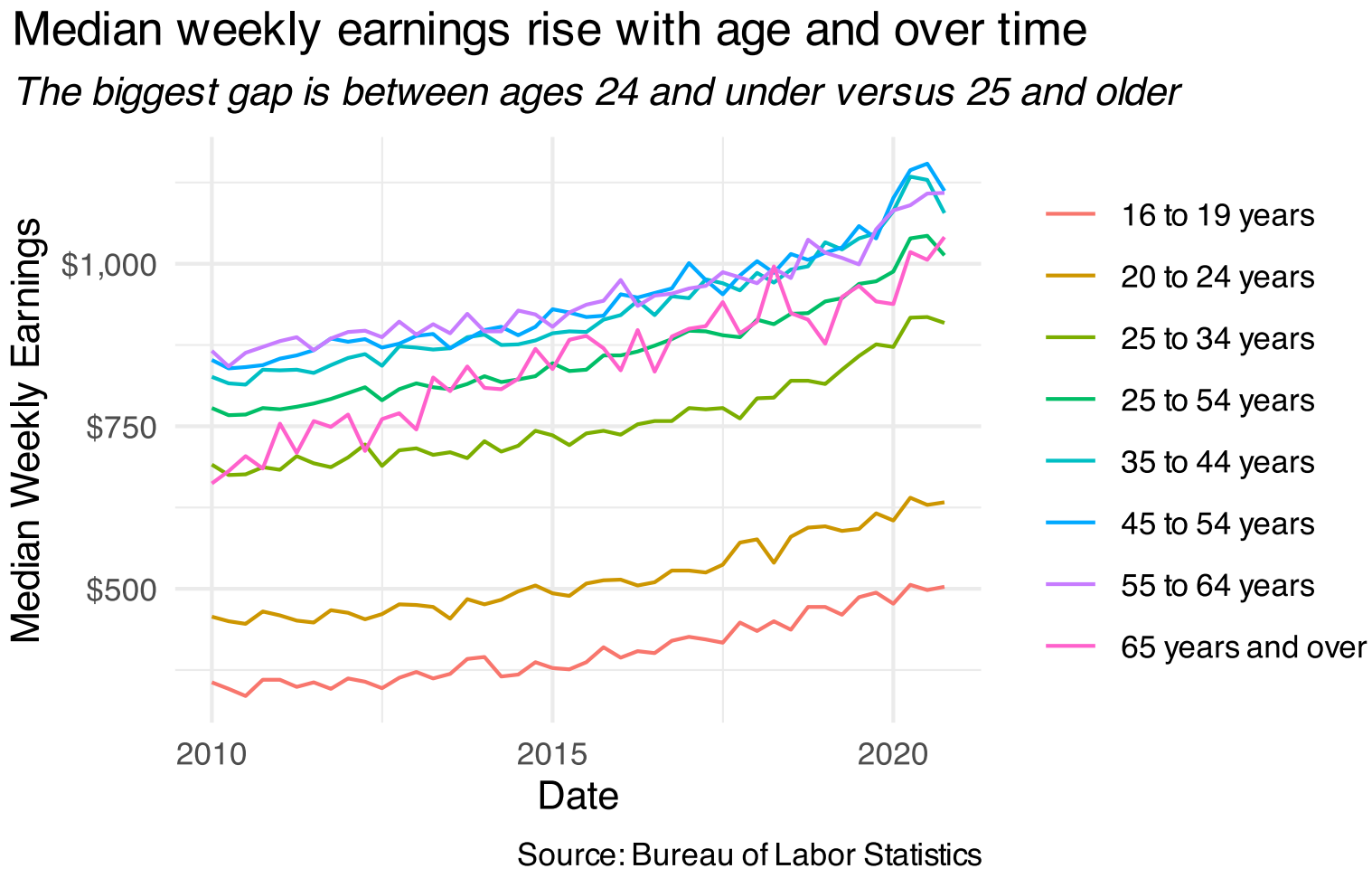


Source: Bureau of Labor Statistics

Adjusting Scales

Scale functions control an axis, the color scale, or the size scale. One common fix: formatting dollar amounts.

```
age_plot <- age_plot +
  scale_y_continuous(labels = scales::dollar_format(accuracy = 1))
age_plot
```



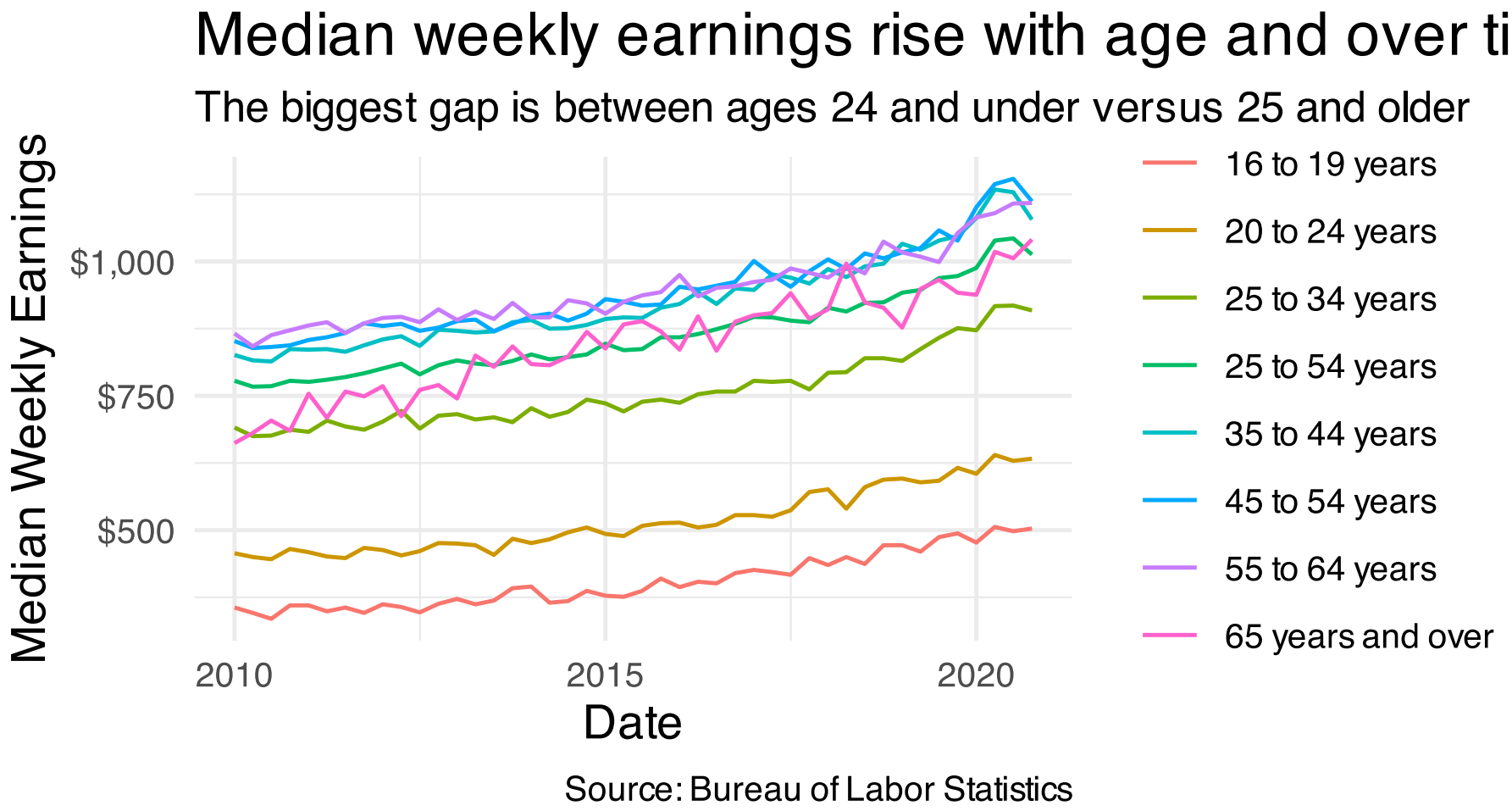
- Earlier, we used `scale_x_log10()` on the Gapminder plot — the same idea, for a different axis

Adjusting Font Size

Font size is one of the most important (and most finicky) changes you can make. The reliable recipe:

1. Set `base_size` inside your `theme_*()` call
2. Find the element you want to adjust (`plot.title`, `axis.title`, …)
3. Set its size with `element_text(size = rel(#))` — `rel()` scales *relative* to `base_size`

```
age_plot +
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(size = rel(1.4)),
        axis.title = element_text(size = rel(1.1)))
```



- This is genuinely hard to memorize — lean on the `ggplot` cheat sheet and online examples rather than trying to hold it all in your head

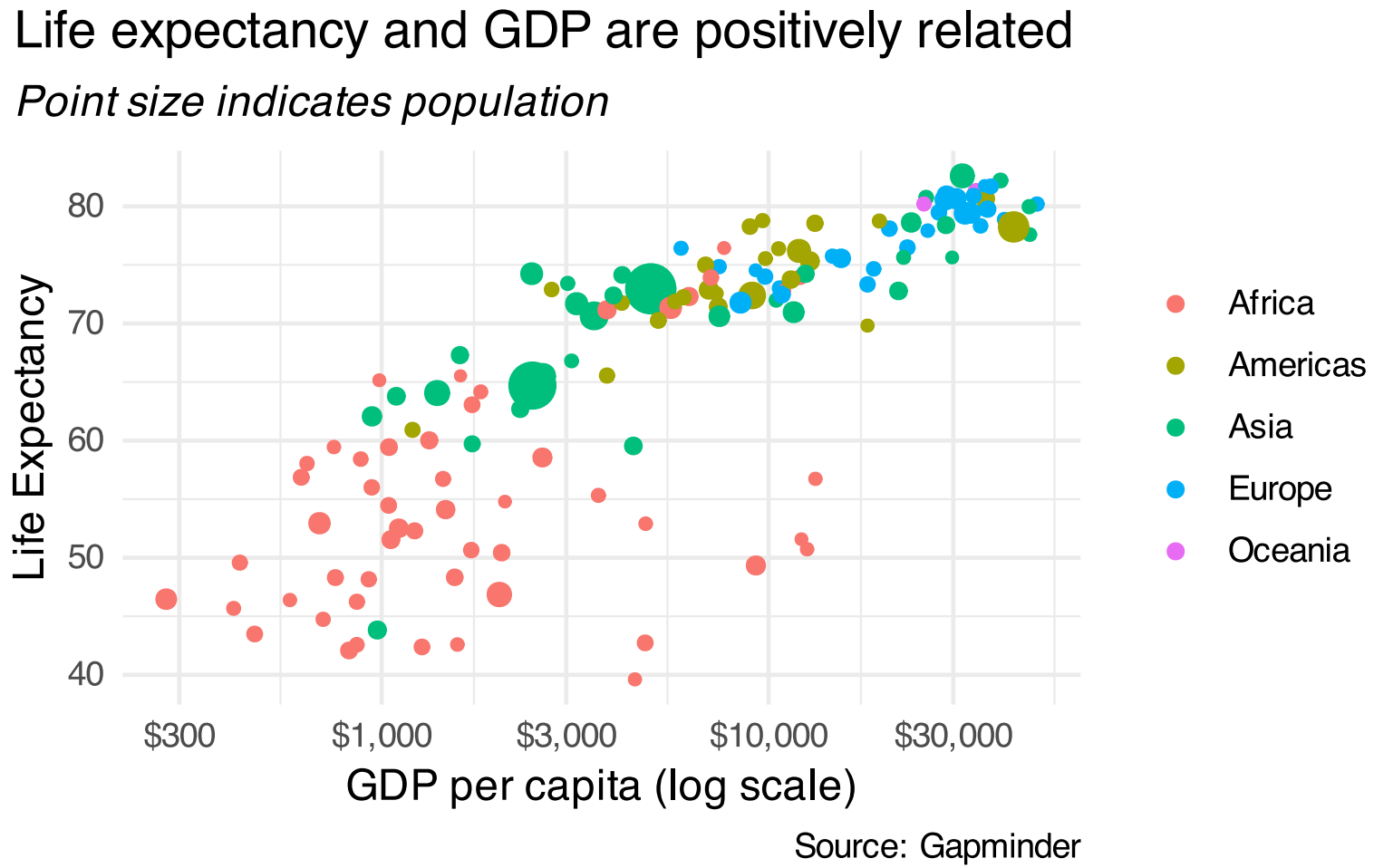
Breakout: Beautify Your Gapminder Plot (5 minutes)

Go back to your Gapminder plot from the last breakout and:

1. Add a title, subtitle, and caption with `labs()`.
2. Add a pre-made theme.
3. Format the x-axis (GDP per capita) as a dollar scale.

Solution

```
gapminder %>%
  filter(year == 2007) %>%
  ggplot(aes(gdpPercap, lifeExp, color = continent, size = pop)) +
  geom_point() +
  scale_x_log10(labels = scales::dollar_format(accuracy = 1)) +
  scale_size(guide = NULL) +
  labs(title = "Life expectancy and GDP are positively related",
       subtitle = "Point size indicates population",
       x = "GDP per capita (log scale)", y = "Life Expectancy",
       caption = "Source: Gapminder", color = NULL) +
  theme_minimal(base_size = 12) +
  theme(plot.title.position = "plot",
        plot.subtitle = element_text(face = "italic", hjust = 0))
```



- Compare this to the very first version we plotted, with no mapping beyond x and y and no labels at all — same data, same `geom_point()`, far easier to read

04

Looking for Relationships: The Redfin Data

01 Recap & Agenda
02 The Grammar of Graphics
03 Beautifying Plots
04 Looking for Relationships: The Redfin Data
05 Regression I
06 Economic vs. Statistical Significance
07 Research Process & The Project
08 Recap

New Data: Redfin Home Sales

A hypothetical: a city is planning an affordable housing project, and wants to know what's actually associated with home prices in the DC metro area.

- We'll turn to Redfin listing data to look for patterns before we draw any conclusions

The six questions, applied to this data:

- **Who:** homes bought or sold in the data
- **Where:** DC, Maryland, and Virginia, in the DC metro area
- **What:** property characteristics, sale price, sale date
- **When:** September 2017
- **Why:** Redfin maintains a public database of real estate listings and sales
- **How:** entered by real estate agents as homes are listed and sold

What's in the Redfin Data?

```
glimpse(redfin)
```

```
Rows: 1,173
Columns: 18
$ id             <chr> "11272669", "21971541", "12042691", "23385384", "112675…
$ address        <chr> "1830 Columbia Pike #205", "4139 Four Mile Run Dr S #20…
$ sale_type      <chr> "PAST SALE", "PAST SALE", "PAST SALE", "PAST SALE", "PA…
$ sold_date      <date> 2017-09-29, 2017-09-06, 2017-09-19, 2017-09-29, 2017-0…
$ list_date      <date> 2017-09-26, 2017-08-11, 2017-09-06, 2017-09-26, 2017-0…
$ property_type  <chr> "Condo/Co-op", "Condo/Co-op", "Condo/Co-op", "Condo/Co-…
$ city           <chr> "Arlington", "Arlington", "Arlington", "Arlington", "Ar…
$ state          <chr> "VA", "VA", "VA", "VA", "VA", "VA", "VA", "VA", "VA", "…
$ zip_code       <dbl> 22204, 22204, 22202, 22204, 22206, 22202, 22206, 22202,…
$ price          <dbl> 280000, 405000, 510000, 395000, 339900, 415000, 319000,…
$ beds           <dbl> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 0, 0, 1…
$ baths          <dbl> 1.0, 2.0, 2.0, 2.0, 1.0, 2.5, 1.0, 2.0, 2.0, 2.0, 2.0, …
$ square_feet    <dbl> 1055, 1030, 1209, 1135, 930, 1606, 800, 1164, 1253, 100…
$ ppsf           <dbl> 265.4028, 393.2039, 421.8362, 348.0176, 365.4839, 258.4…
$ lot_size       <dbl> 1055, 1030, 1209, 1135, 930, 1606, 800, 1164, 1253, 100…
$ year_built     <dbl> 1964, 1966, 2006, 1966, 1983, 1976, 1950, 2006, 1965, 1…
$ status         <chr> "Sold", "Sold", "Sold", "Sold", "Sold", "Sold", "Sold",…
$ metro_distance <dbl> 0.86720529, 2.18504165, 0.79187050, 2.21886788, 2.44636…
```

- If we wanted to predict a home's price, which of these look like promising predictors?

Correlation Matrix of the Redfin Data

Recall from Lecture 6: `cor()` returns every pairwise correlation when passed more than two columns.

```
redfin %>%
  select(price, beds, baths, square_feet, ppsf, lot_size, metro_distance) %>%
  cor(use = "pairwise.complete.obs") %>%
  round(2) %>%
  knitr::kable()
```

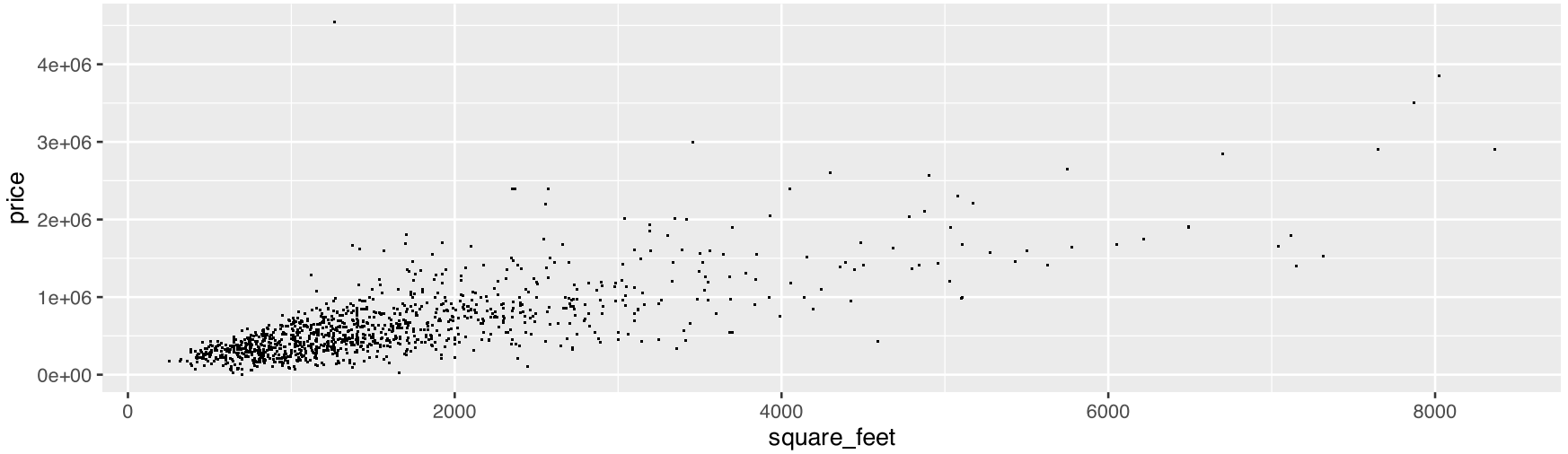
	price	beds	baths	square_feet	ppsf	lot_size	metro_distance
price	1.00	0.56	0.71	0.74	0.45	0.40	-0.04
beds	0.56	1.00	0.80	0.76	-0.19	0.51	0.18
baths	0.71	0.80	1.00	0.84	-0.07	0.50	0.18
square_feet	0.74	0.76	0.84	1.00	-0.17	0.58	0.19
ppsf	0.45	-0.19	-0.07	-0.17	1.00	-0.12	-0.38
lot_size	0.40	0.51	0.50	0.58	-0.12	1.00	0.37
metro_distance	-0.04	0.18	0.18	0.19	-0.38	0.37	1.00

Reading the Matrix

- Which variables are most strongly correlated with `price`? Which are only weakly correlated?
- `beds`, `baths`, and `square_feet` are all strongly correlated with `price` — but also strongly correlated with *each other*. They're all really measuring the same thing: how big the home is.
- `ppsf` (price per square foot) normalizes for size, and correlates more clearly with `metro_distance`
- A correlation matrix is a quick way to flag which variables might matter *before* you commit to a regression — and to catch a coding mistake (an unexpectedly extreme correlation is often a sign something's wrong)

Visualizing Price and Square Footage: First Pass

```
ggplot(redfin, aes(x = square_feet, y = price)) +
  geom_point(shape = ".")
```

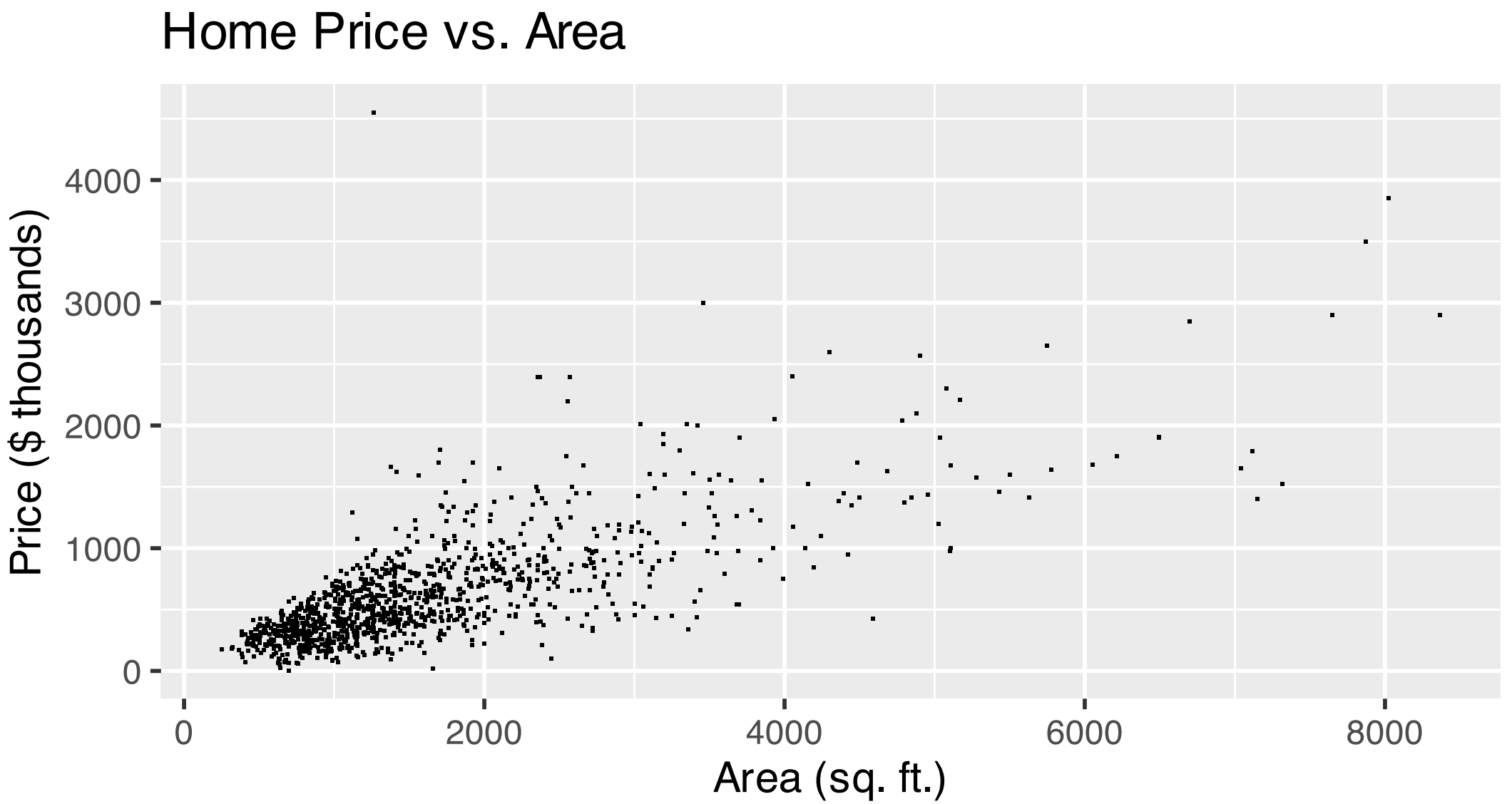


- The outcome we care about (`price`) goes on the y-axis
- The y-axis is hard to read as-is — let's rescale it

Scaling and Labeling the Axes

```
redfin_plot <- ggplot(redfin, aes(x = square_feet, y = price / 1000)) +
  geom_point(shape = ".") +
  labs(x = "Area (sq. ft.)", y = "Price ($ thousands)",
       title = "Home Price vs. Area")

redfin_plot
```


Home Price vs. Area
Price ($ thousands)
4000
3000
2000
1000
0
0
2000
4000
6000
8000
Area (sq. ft.)

What Does This Chart Tell Us?

- Most homes are small and cheap, relative to the handful of large, expensive ones on the right
- Moving left to right, prices increase *on average* — the relationship doesn't have to be a perfect line to still be useful
- How would you summarize "price increases with square footage, on average" as a single number? That's exactly what a regression line does.

05

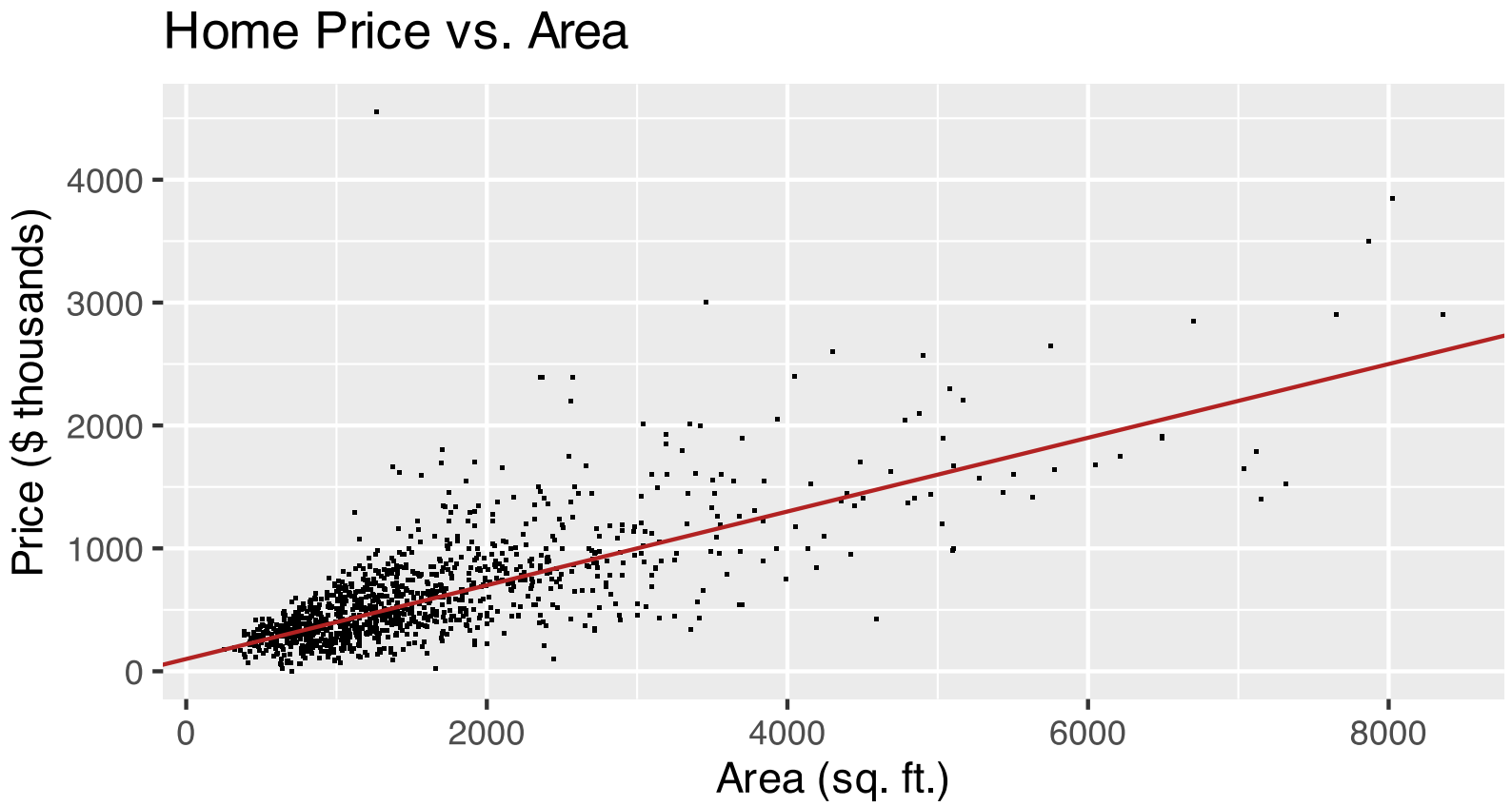
Regression I

01 Recap & Agenda
02 The Grammar of Graphics
03 Beautifying Plots
04 Looking for Relationships: The Redfin Data
05 Regression I
06 Economic vs. Statistical Significance
07 Research Process & The Project
08 Recap

Disclaimer

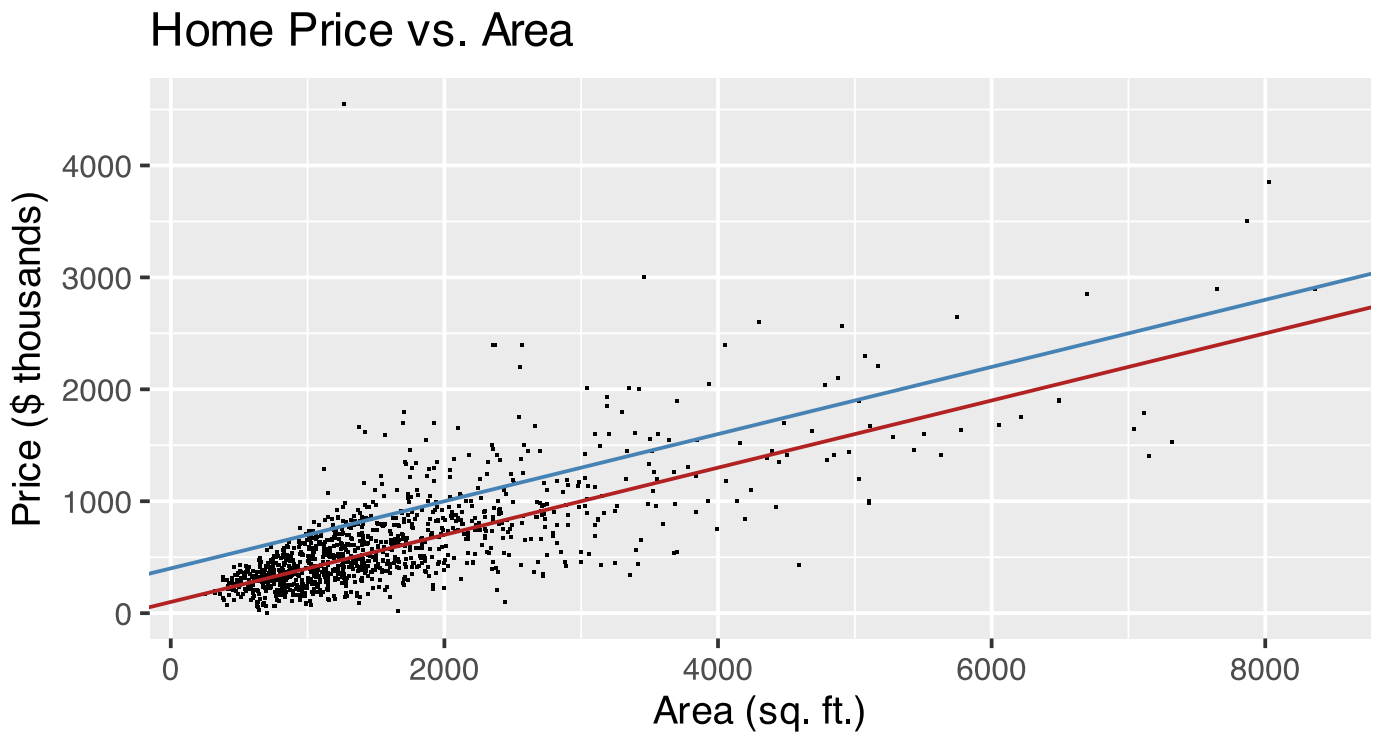
- This isn't an econometrics class — the goal is interpretation and communication, not derivation
- Ask your economist adviser if you want the deeper theory behind what follows
- All we're doing today: finding the line that "best summarizes" a relationship

Picking a Line by Eye



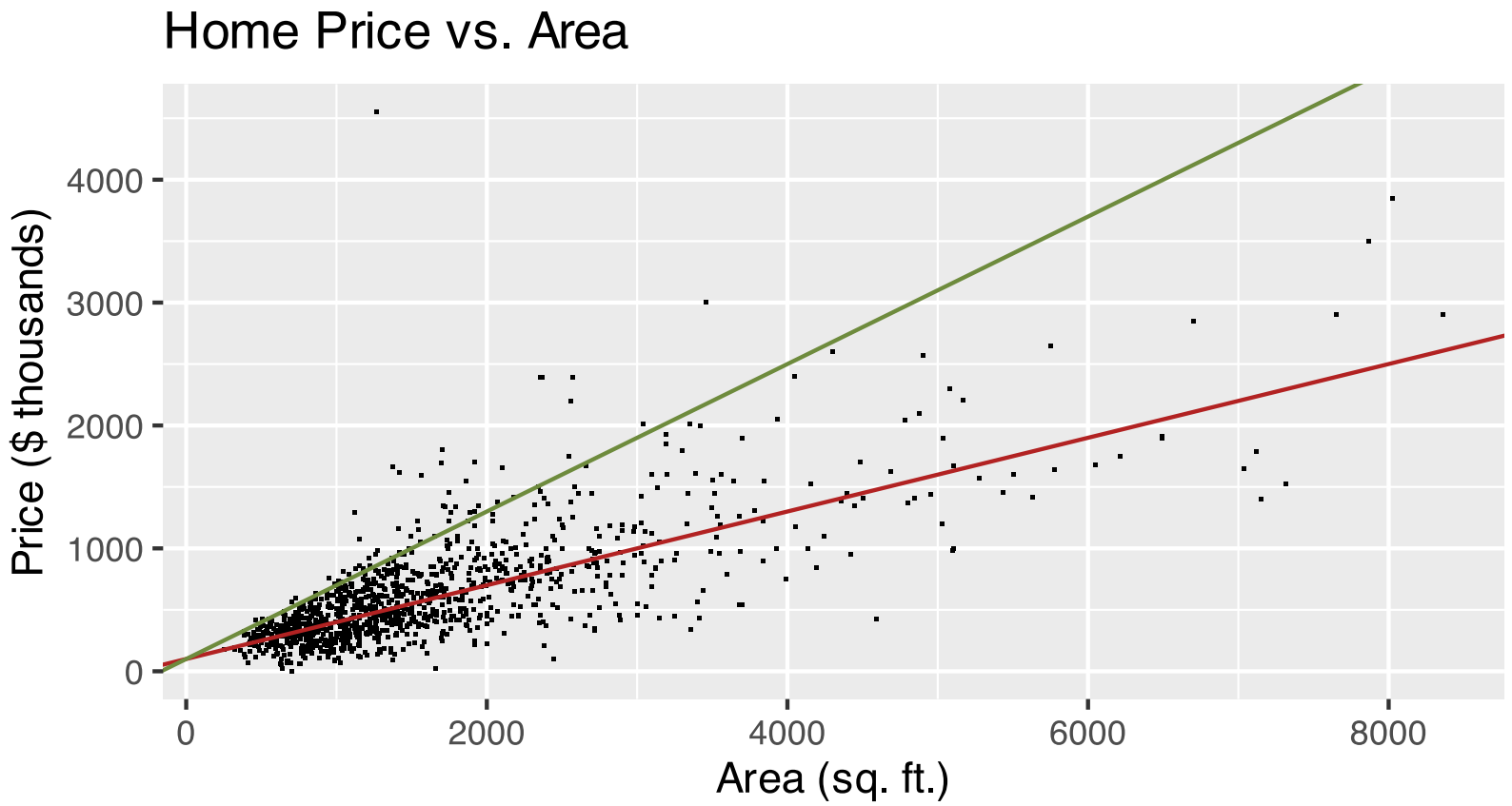
- Does this look like a reasonable summary of the relationship?

A Different Intercept



- Changing the intercept shifts every predicted price up or down by the same amount — the *rate* of increase (the slope) hasn't changed

A Different Slope



- A steeper slope means price increases *faster* as square footage increases

Quantifying “Best”

- We could keep guessing lines by eye, but that’s slow and nobody will agree on which one is “best”
- We need a rule that consistently picks one line, given the data
- A linear regression picks the line that makes the squared vertical distance between each point and the line as small as possible, summed across every point — **minimizing the sum of squared errors**

The Regression Equation

$$y = mx + b \qquad \rightarrow \qquad \text{Price} = m \times \text{SquareFeet} + b$$

- x is our **explanatory** (independent) variable
- y is our **outcome** (dependent) variable
- b, the intercept, is the predicted value of y when $x = 0$
- m, the slope, is how much y changes for a one-unit increase in x

Running a Regression in R: `lm()`

```
## general case
lm(y ~ x, data = my_data)
```

```
price_mod <- lm(price ~ square_feet, data = redfin)
```

- The variable to the *left* of `~` is the outcome; the variable to the *right* is the explanatory variable
- Always specify `data =` — otherwise R has no idea which `price` and `square_feet` you mean

Getting Results Out of `price_mod`: The Tidy Way

`price_mod` is an R model object — not a data frame. You could reach for `summary(price_mod)` and then dig a coefficient out with `$`, but that's exactly the pattern we've been avoiding all semester.

`broom` turns a model into tidy tibbles instead:

- `broom::tidy()` — one row per coefficient (estimate, standard error, test statistic, p-value)
- `broom::glance()` — one row of model-level fit statistics (R^2, overall p-value, sample size)
- `broom::augment()` — the original data, plus fitted values and residuals, one row per observation

Because each of these returns a normal tibble, every verb you already know — `select()`, `filter()`, `pull()`, `kable()` — works on the output.

Coefficients With `broom::tidy()`

```
tidy(price_mod) %>%
  mutate(p.value = scales::pvalue(p.value)) %>%
  knitr::kable(digits = 2)
```

term	estimate	std.error	statistic	p.value
(Intercept)	120650.96	16159.24	7.47	<0.001
square_feet	312.67	8.30	37.68	<0.001

- `estimate` is the economic relationship — the coefficients m and b
- `statistic` and `p.value` are what we'll use for statistical significance in a few slides

Model Fit With `broom::glance()`

```
glance(price_mod) %>%
  select(r.squared, p.value, nobs) %>%
  knitr::kable(digits = 3)
```

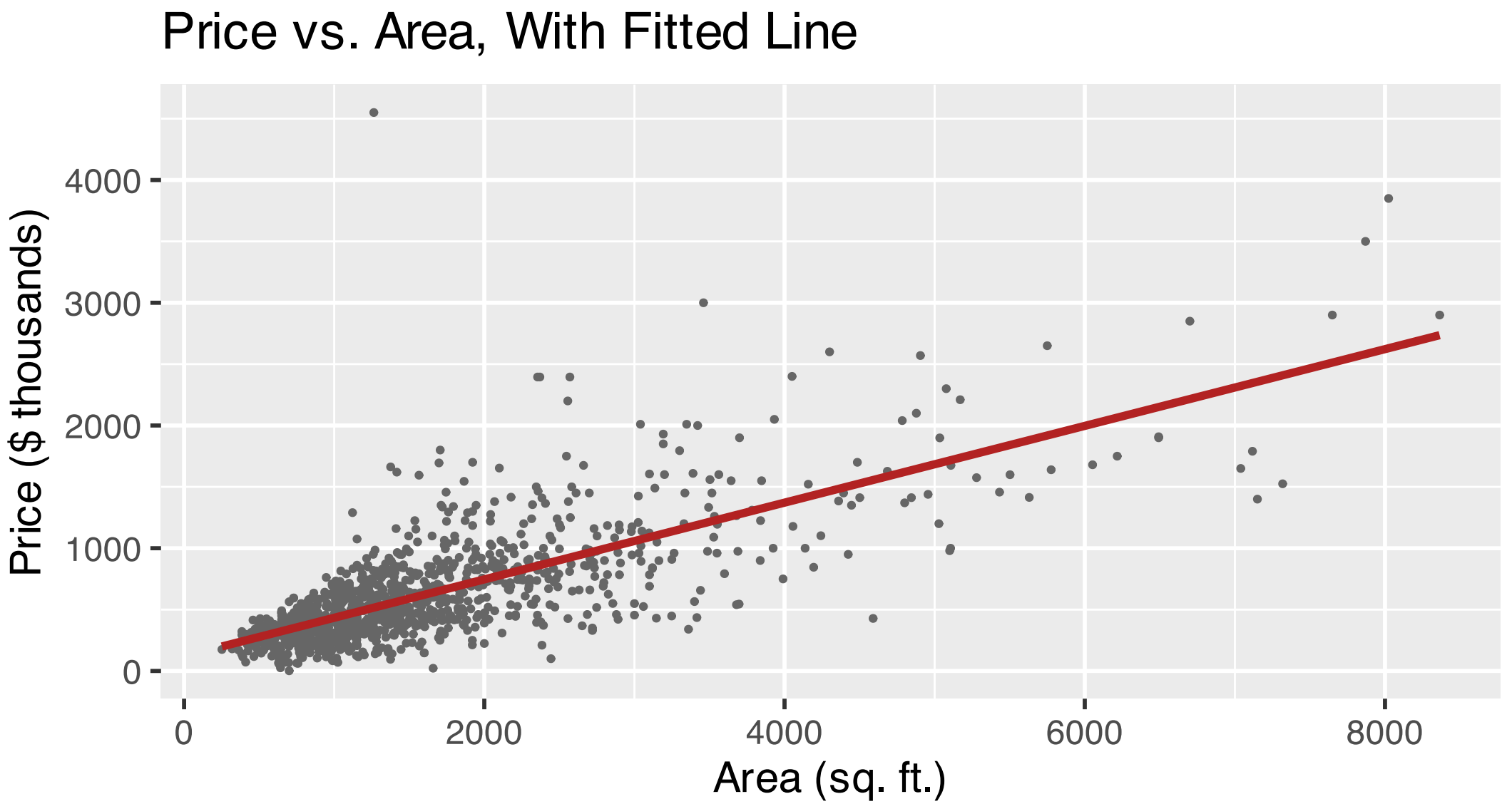
r.squared	p.value	nobs
0.551	0	1161

- `r.squared`: the share of variation in price "explained" by square footage
- `nobs`: how many rows were actually used (`lm()` silently drops rows with missing values)

Visualizing the Regression Line

`broom::augment()` adds a `.fitted` column — the model's predicted value for every row — right onto the original data. Plot it with `geom_line()`, no manual intercept or slope needed:

```
augment(price_mod) %>%
  ggplot(aes(x = square_feet, y = price / 1000)) +
  geom_point(size = 0.4, color = "grey40") +
  geom_line(aes(y = .fitted / 1000), color = "firebrick", linewidth = 1) +
  labs(x = "Area (sq. ft.)", y = "Price ($ thousands)",
       title = "Price vs. Area, With Fitted Line")
```


Price vs. Area, With Fitted Line
Price ($ thousands)
4000
3000
2000
1000
0
0
2000
4000
6000
8000
Area (sq. ft.)

Interpreting the Coefficients

```
tidy(price_mod) %>%
  mutate(p.value = scales::pvalue(p.value)) %>%
  knitr::kable(digits = 2)
```

term	estimate	std.error	statistic	p.value
(Intercept)	120650.96	16159.24	7.47	<0.001
square_feet	312.67	8.30	37.68	<0.001

1. What does the model predict for the price of a home with 0 square feet? Does that number mean anything on its own?
2. For every additional square foot, how much does predicted price change?
3. Why does interpreting the intercept require some judgment, even though R reports it precisely?

What Do the Numbers in `tidy()` Mean?

- `estimate` tells us the *magnitude* of the relationship
- But how sure are we that the relationship isn't just noise? That's a hypothesis test — the same four steps from Lecture 6

Hypothesis Testing a Coefficient

1. Form the null and alternative hypotheses
2. Set a significance level α
3. Run the test, and read off the p-value — `tidy()` already computed it for you
4. Reject or fail to reject H_0 by comparing the p-value to α

- $p \leq \alpha$: reject H_0 — the coefficient is **statistically significant**
- $p > \alpha$: fail to reject — we don't have enough evidence that the coefficient differs from zero

The Null Hypothesis on a Slope

For the relationship between square footage and price, the natural null hypothesis is "no relationship" — a slope of zero:

$$H_0 : m = 0 \qquad H_a : m \neq 0$$

```
tidy(price_mod) %>%
  mutate(p.value = scales::pvalue(p.value)) %>%
  knitr::kable(digits = 2)
```

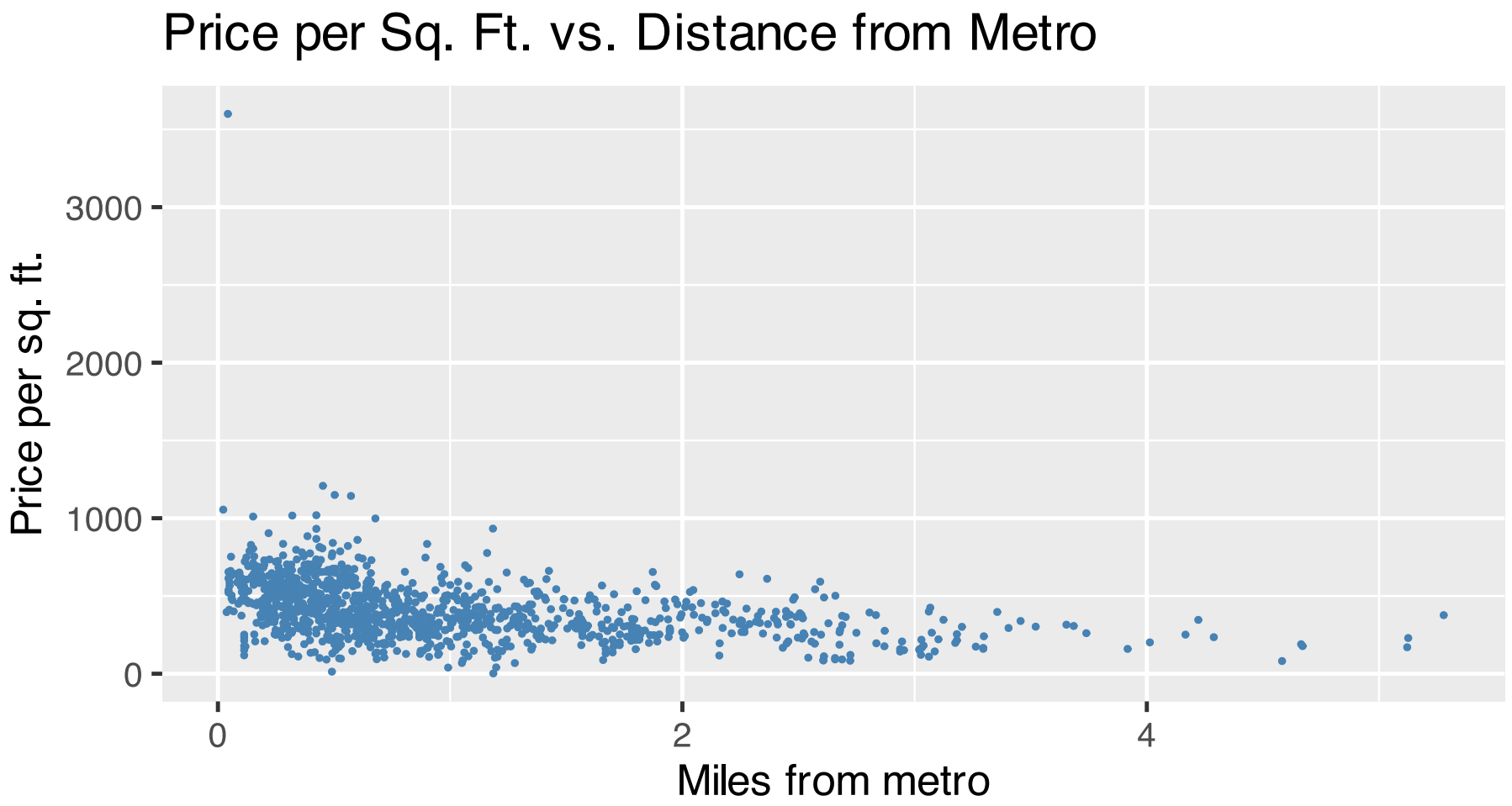
term	estimate	std.error	statistic	p.value
(Intercept)	120650.96	16159.24	7.47	<0.001
square_feet	312.67	8.30	37.68	<0.001

- At $\alpha = 0.05$, do we reject H_0 for `square_feet`? How do you know, just from this table?
- Economists often mark $p \leq 0.1, 0.05, 0.01$ with *, **, *** — but `tidy()`'s actual `p.value` column tells you more than stars alone

Class Exercise: A New Relationship

How might distance from the metro affect price *per square foot* (`ppsf`), independent of a home's size? What sign and magnitude would you guess?

```
ggplot(redfin, aes(x = metro_distance, y = ppsf)) +
  geom_point(size = 0.3, color = "steelblue") +
  labs(x = "Miles from metro", y = "Price per sq. ft.",
       title = "Price per Sq. Ft. vs. Distance from Metro")
```



- Notice anything strange? Regressions are sensitive to outliers — one point here is far larger than the rest.

Tracking Down the Outlier

```
redfin %>%
  filter(ppsf > 2000) %>%
  select(id, price, square_feet, ppsf)
```

```
# A tibble: 1 × 4
  id         price square_feet  ppsf
  <chr>      <dbl>       <dbl> <dbl>
1 10349836 4550000        1264 3600.
```

- A quick look at the actual Redfin listing shows the sale price was entered incorrectly — off by a factor of 10. We correct it rather than drop the observation, since the rest of the row (`square_feet`, `metro_distance`) is still good data:

```
redfin <- redfin %>%
  mutate(ppsf = if_else(id == "10349836", ppsf / 10, ppsf))
```

- This is also why we look at our data *before* regressing on it — a bad regression often starts as a bad row

Breakout: Regress and Plot (10 minutes)

1. Regress `ppsf` on `metro_distance` with `lm()`.
2. Pull the coefficients with `broom::tidy()`.
3. Add the fitted line to the scatterplot using `broom::augment()`.
4. What's the relationship between metro distance and price per square foot?

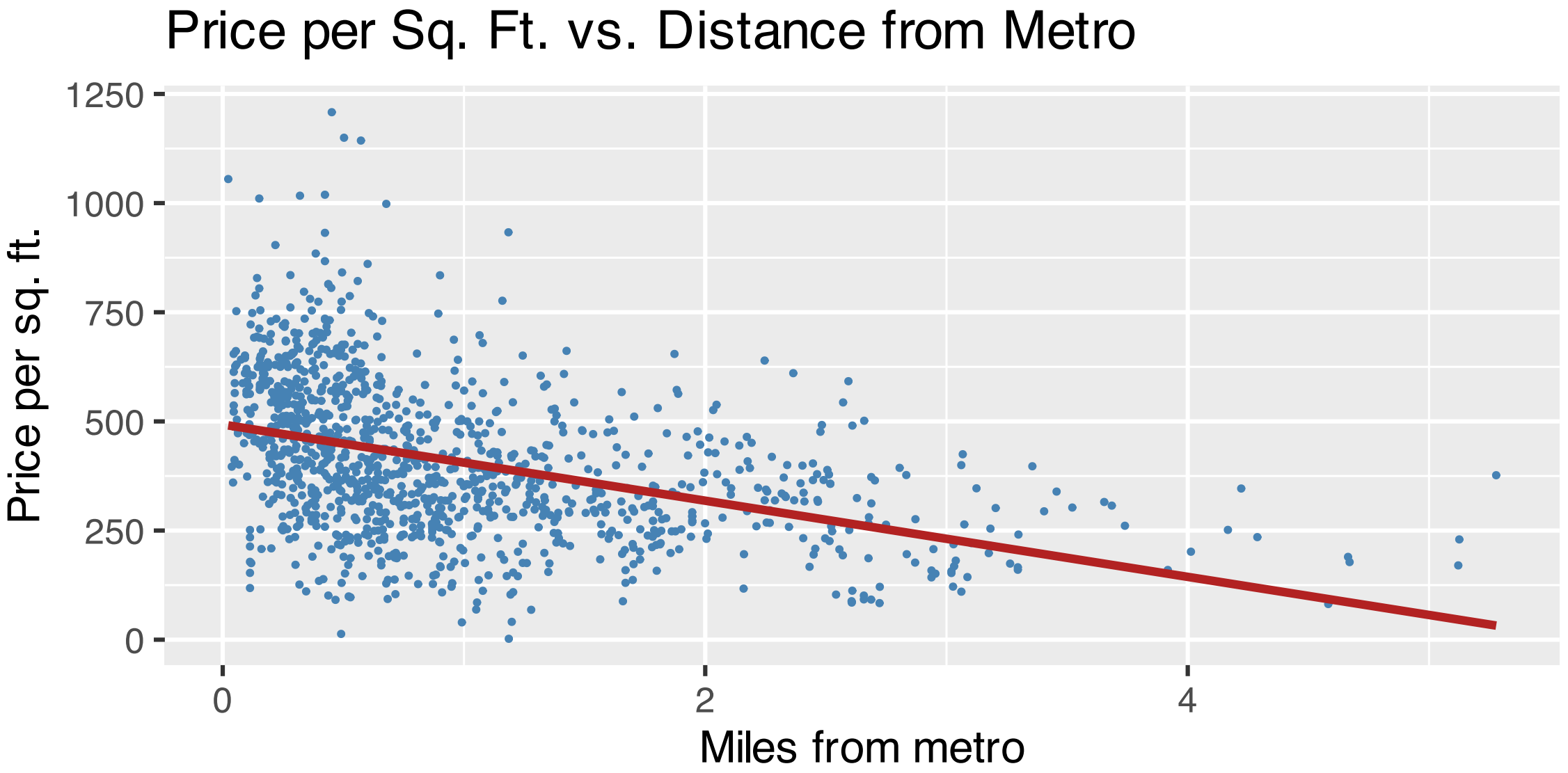
Solution

```
metro_mod <- lm(ppsf ~ metro_distance, data = redfin)

tidy(metro_mod) %>%
  mutate(p.value = scales::pvalue(p.value)) %>%
  knitr::kable(digits = 2)
```

term	estimate	std.error	statistic	p.value
(Intercept)	492.83	7.12	69.19	<0.001
metro_distance	−87.24	5.58	−15.64	<0.001

```
augment(metro_mod) %>%
  ggplot(aes(x = metro_distance, y = ppsf)) +
  geom_point(size = 0.3, color = "steelblue") +
  geom_line(aes(y = .fitted), color = "firebrick", linewidth = 1) +
  labs(x = "Miles from metro", y = "Price per sq. ft.",
       title = "Price per Sq. Ft. vs. Distance from Metro")
```


Price per Sq. Ft. vs. Distance from Metro
Price per sq. ft.
1250
1000
750
500
250
0
0
2
4
Miles from metro

Discussion

- The intercept predicts about $493/sq. ft. right on top of a metro station; each additional mile away is associated with about an $87 *decrease* in price per square foot
- Is this statistically significant? Is it economically significant — recall the mean `ppsf` is a bit over $400
- We're describing an association here, not a causal effect — more on that distinction in a later lecture

06

Economic vs. Statistical Significance

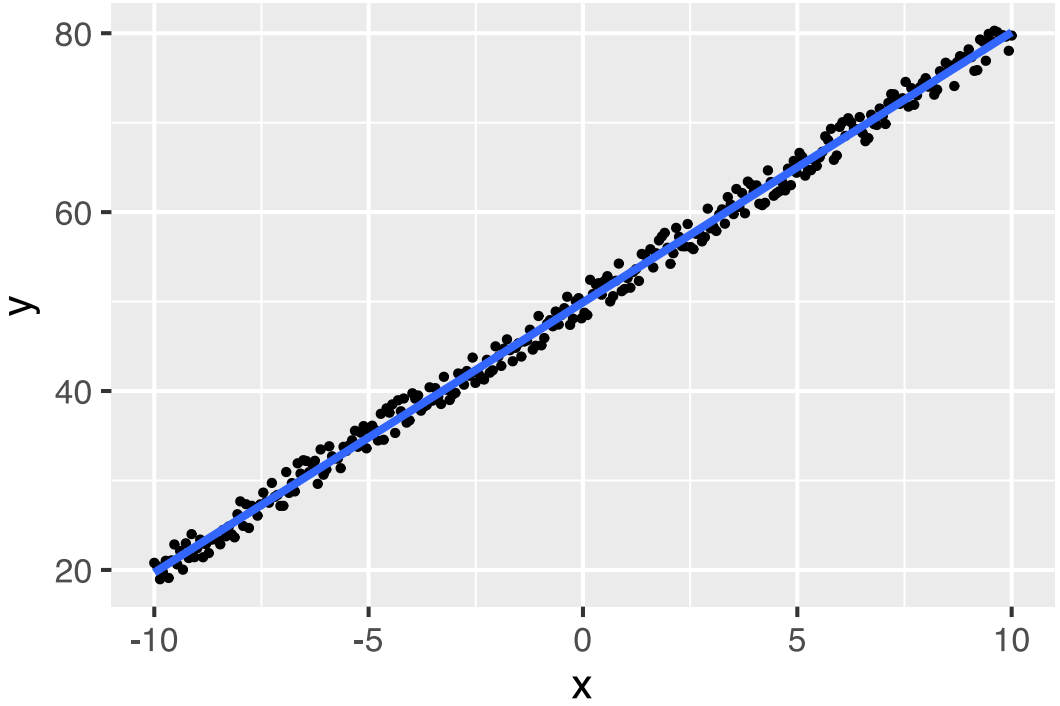
01 Recap & Agenda
02 The Grammar of Graphics
03 Beautifying Plots
04 Looking for Relationships: The Redfin Data
05 Regression I
06 Economic vs. Statistical Significance
07 Research Process & The Project
08 Recap

Two Different Questions

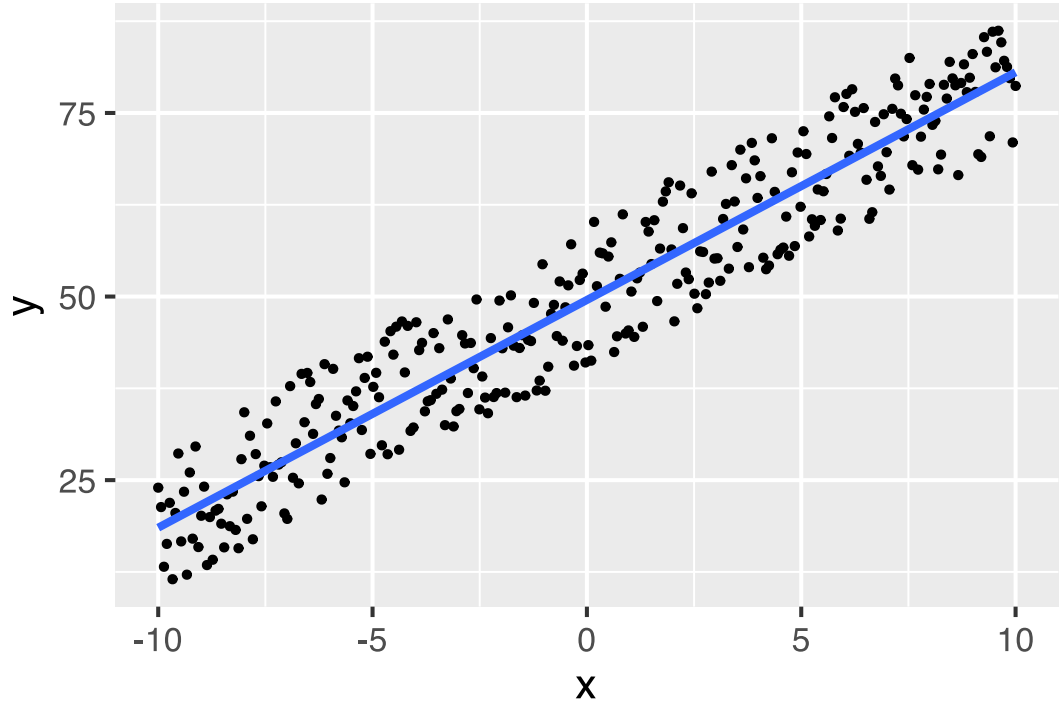
- **Statistical significance (precision):** is the relationship unlikely to be due to chance, given this sample? This is what the p-value answers.
- **Economic significance (magnitude):** is the relationship *large enough to matter*, given the real-world context? There's no R function for this — it's judgment.

Statistical Significance: More vs. Less Precise

More Precise



Less Precise



Same true slope ($m = 3$) in both — more noise around the line means a larger standard error, a smaller test statistic, and a bigger p-value.

Economic Significance: Judgment, Not a Function

- Something can be economically significant but *not* statistically significant — a real effect, measured too imprecisely to be sure of (often: too small a sample)
- Something can be statistically significant but economically **insignificant** — a tiny, real effect, measured so precisely (often: a huge sample) that we're confident it's not exactly zero, even though it barely matters
- Context decides what "matters." If `ppsf` fell by 25 cents for every extra mile from the metro, is that a big deal when average `ppsf` is $30? What about when it's $990? **Check your summary statistics before judging magnitude.**

07

Research Process & The Project

01 Recap & Agenda
02 The Grammar of Graphics
03 Beautifying Plots
04 Looking for Relationships: The Redfin Data
05 Regression I
06 Economic vs. Statistical Significance
07 Research Process & The Project
08 Recap

Research Process: Project Update #6

Project Update #6 asks you to:

- **Iterate** on Project Update #5 — revise it based on the feedback you received
- **Refine your interpretation** again, now that you have another week of analysis behind you
- Run a **simple regression** on a relationship in your own data, and pull its results with `broom`

As always, you'll also follow up with your adviser and post an update for your classmates.

This Week's Timeline

- **HW #8 — due October 29.** Today's material: advanced plots and Regression I.
- **Project Update #6 — due October 31.** Iterating on feedback, refined interpretation, and a simple regression on your own data.

Think now about which two variables in your own data you'd regress on each other — you'll want a dependent variable and one explanatory variable in mind before you sit down to write Update #6.

08

Recap

01 Recap & Agenda
02 The Grammar of Graphics
03 Beautifying Plots
04 Looking for Relationships: The Redfin Data
05 Regression I
06 Economic vs. Statistical Significance
07 Research Process & The Project
08 Recap

Recap

- The **Grammar of Graphics** still underlies every `ggplot` — map data to aesthetics, represent it with a geometry
- **Beautifying** a plot (`labs()`, themes, scales) is what turns an exploratory chart into one you can hand to someone else
- `lm(y ~ x, data = ...)` fits a linear regression — the line that minimizes the sum of squared errors
- `broom::tidy()`, `broom::glance()`, and `broom::augment()` are the tidy way to get coefficients, model fit, and fitted values out of a model — no `$`, no `summary(model)$coefficients`
- **Statistical significance** (is it zero?) and **economic significance** (does it matter?) are different questions, and you need both

Functions Introduced Today

- `labs()`, `theme_*()`, `theme()`, `scale_*()` — plot titles, pre-made themes, and axis/legend formatting
- `lm()` — fit a linear regression
- `broom::tidy()`, `broom::glance()`, `broom::augment()` — tidy regression output

Documentation & Further Reading

- `?labs`, `?theme`, `?scale_x_continuous` — plot customization
- `?lm`, `?broom::tidy`, `?broom::glance`, `?broom::augment` — fitting and reading a regression
- https://www.r-graph-gallery.com/ — the `ggplot` gallery, for inspiration
- https://broom.tidymodels.org/ — the full list of models `broom` supports

Coming up: Regression II — adding more explanatory variables.